Real-world — a support-reply router — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `runnable_routing_practice.py`: a real support desk doesn't answer every ticket the same way — a billing question, a bug report, and a random hello each need a different reply. This exercise builds exactly that, and introduces the two LCEL pieces that make it work. A `RunnableBranch` is an if/elif/else for chains: you give it `(condition, chain)` pairs and it sends each input to the first chain whose condition is true, with a default for everything else — so each category gets its own dedicated reply chain. A `RunnableLambda` wraps a plain Python function so it can sit inside a chain like any other step; here it tags the routed result so you can see which branch handled it. Together they turn "pick the right responder, then label the output" into one composable chain. **If not:** Project 6's ticket-triage pipeline (and Doc11's Supervisor) would be the first time you met routing with `RunnableBranch`, in a much bigger file.

Solution

`build_pipeline` is written once and used twice — with a real model below, and with a fake model in the test. That reuse is what lets you prove the routing with no model and no key: a fake model that echoes the system line it was given tells you which branch ran.

```
# runnable_routing_practice.py
from langchain_core.messages import AIMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableBranch, RunnableLambda
from langchain_openai import ChatOpenAI


# A reply chain for one system prompt.
def reply_chain(model, system_text):
    prompt = ChatPromptTemplate.from_messages(
        [("system", system_text), ("human", "{text}")]
    )
    return prompt | model | StrOutputParser()


# A little tag added after whichever branch answered.
def tag_reply(text):
    return f"{text.strip()} (auto-routed)"
```

```
# Build the whole routing pipeline for a given model.
def build_pipeline(model):
    billing = reply_chain(model, "You are a billing agent. Mention refunds.")
    technical = reply_chain(model, "You are tech support. Ask for repro steps.")
    general = reply_chain(model, "You are general support. Friendly, brief.")
    # RunnableBranch takes the first branch whose test is True, else the last.
    router = RunnableBranch(
        (lambda t: t["category"] == "billing", billing),
        (lambda t: t["category"] == "technical", technical),
        general,
    )
    return router | RunnableLambda(tag_reply)


# --- Run it against a real model ---
real_model = ChatOpenAI(model="gpt-4o-mini", temperature=0, timeout=30)
pipeline = build_pipeline(real_model)
for category in ["billing", "technical", "hello"]:
    reply = pipeline.invoke({"category": category, "text": "help me"})
    print(f"{category} -> {reply}")


# --- Prove the routing with no key: a fake model that echoes its system
# line, so the reply tells us which branch ran. ---
def echo_system_line(prompt_value):
    return AIMessage(content=prompt_value.to_messages()[0].content)


test_pipeline = build_pipeline(RunnableLambda(echo_system_line))
billing = test_pipeline.invoke({"category": "billing", "text": "x"})
technical = test_pipeline.invoke({"category": "technical", "text": "x"})
unknown = test_pipeline.invoke({"category": "sales", "text": "x"})
print("case 1 (billing -> billing branch) ->", "billing agent" in billing)
print("case 1 (technical -> tech branch)  ->", "tech support" in technical)
print("case 2 (unknown -> default + tag)  ->",
      "general support" in unknown and "(auto-routed)" in unknown)
```

Expected output (the tests, no key needed):

```
case 1 (billing -> billing branch) -> True
case 1 (technical -> tech branch) -> True
case 2 (unknown -> default + tag) -> True
```

What you proved: the three chain shapes together — sequential (`|`), your own function (`RunnableLambda`), and conditional (`RunnableBranch`) — in one real pipeline, with the routing checked for free.